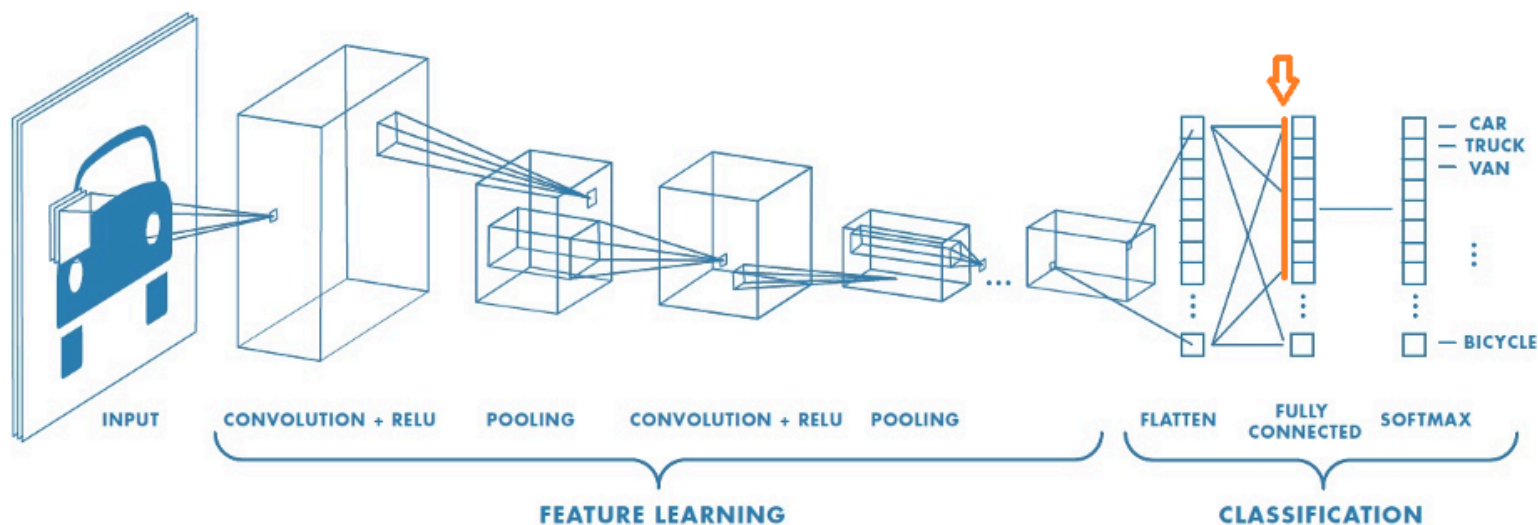


✓ Векторные представления изображений

Как мы убедились, информация, содержащаяся в промежуточных слоях хорошо обученных нейронных сетей, достаточна для классификации изображений. Это значит, что мы можем заменить изображение этой информацией и работать с ней. Например, с предпоследнего слоя получается некоторый вектор, значит можем исходное изображение представить (описать) этим вектором.

Получили векторное представление изображения. Эти вектора не банальны, их достаточно, чтобы классифицировать изображения, значит в них установлены какие-то важные взаимосвязи между пикселями изображения.

А давайте такие вектора кластеризуем. Не окажется ли так, что они группируются в "похожие" группы. Для кластеризации и уменьшения размерности можно воспользоваться методами главных компонент PCA и T-SNE, давайте с их помощью отобразим вектора изображений в двумерное пространство.



Возьмем уже обученную сеть (здесь ResNet50).

Пропустим через нее изображение. И возвратим вектор, который получается перед классификатором (здесь выход с полносвязного слоя после слоя flatten, стрелка на рис.). Будем дальше работать с этими векторами.

Все эти операции уже реализованы в библиотеке [img2vec-keras](https://github.com/jaredwinick/img2vec-keras), установим ее.

Для сравнения векторов лучше подходит "косинусное расстояние" - косинус угла между ними, чем больше, тем похожей.

Установим библиотеку и, если потребуется, после установки нажмите "Restart Runtime".

```
!pip install git+https://github.com/jaredwinick/img2vec-keras.git
```



```
Collecting git+https://github.com/jaredwinick/img2vec-keras.git
  Cloning https://github.com/jaredwinick/img2vec-keras.git to /tmp/pip-req-build-66qfsh02
  Running command git clone --filter=blob:none --quiet https://github.com/jaredwinick/img2vec-keras.git
  Resolved https://github.com/jaredwinick/img2vec-keras.git to commit 80c56b2e1b698e36c26c4015e8b21216f1
  Preparing metadata (setup.py) ... done
Requirement already satisfied: numpy>=1.9.1 in /usr/local/lib/python3.10/dist-packages (from img2vec_ker
Requirement already satisfied: tensorflow>=2.0.0 in /usr/local/lib/python3.10/dist-packages (from img2ve
Requirement already satisfied: pillow in /usr/local/lib/python3.10/dist-packages (from img2vec_keras==0
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.10/dist-packages (from tensorflo
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.10/dist-packages (from tensor
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.10/dist-packages (from ter
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.10/dist-pac
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.10/dist-packages (from tens
```

```
Requirement already satisfied: h5py>=3.10.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: ml-dtypes<0.5.0,>=0.3.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<5.0.0dev, in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: tensorboard<2.18,>=2.17 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: keras>=3.2.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.10/dist-packages (from tensorflow>=2.12.0)
Requirement already satisfied: rich in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->tensorflow>=2.12.0)
Requirement already satisfied: namex in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->tensorflow>=2.12.0)
Requirement already satisfied: optree in /usr/local/lib/python3.10/dist-packages (from keras>=3.2.0->tensorflow>=2.12.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.21.0)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.10/dist-packages (from tensorboard>=2.17.0)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.10/dist-packages (from tensorboard>=2.17.0)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from tensorboard>=2.17.0)
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/python3.10/dist-packages (from werkzeug>=1.0.1)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.10/dist-packages (from rich>=13.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.10/dist-packages (from rich>=13.0.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.10/dist-packages (from markdown-it-py>=2.2.0)
Building wheels for collected packages: img2vec_keras
  Building wheel for img2vec_keras (setup.py) ... done
  Created wheel for img2vec_keras: filename=img2vec_keras-0.2-py3-none-any.whl size=6199 sha256=851515fcb0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b0b0
  Stored in directory: /tmp/pip-ephem-wheel-cache-6c12soox/wheels/da/92/ca/893e107e4c49bfd1bb057215620bcb0b0b0b0b0b0b0b0b0b0b0
Successfully built img2vec_keras
Installing collected packages: img2vec_keras
Successfully installed img2vec_keras-0.2
```


From (original): <https://drive.google.com/uc?id=1Madin3qXe3Xexox5q0hASGu1FR2wvsFz>

To: /content/101 ObjectCategories.tar.gz

```
101 ObjectCategories sample data
```

```
from img2vec_keras import Img2Vec # подключаем библиотеку для векторных представлений
```

```
import glob # для работы с файлами
```

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity # похожесть по косинусному расстоянию.
from sklearn.decomposition import PCA # метод главных компонент
from sklearn.manifold import TSNE # метод TSNE
from sklearn.preprocessing import StandardScaler # масштабирование, убирает среднее, единичный разброс

import matplotlib.pyplot as plt
from matplotlib.offsetbox import OffsetImage, AnnotationBbox

import cv2
```

```
img2vec = Img2Vec() # инициализируем вектора, загрузится предобученная сеть.
```

📄 Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels/102967424/102967424 5s 0us/step

```
image_paths = []
image_classes = ['pizza', 'schooner', 'scissors', 'panda', 'brain'] # выберем классы для отображения, провер
```

```
# считываем пути к изображениям этих классов
for image_class in image_classes:
    image_paths.extend(glob.glob('101_ObjectCategories/' + image_class + '/*.jpg')) # только .jpg
```

```
# сколько их
len(image_paths)
```

📄 291

```
# создаем вектора для этих изображений
image_vectors = {}
for image_path in image_paths: # для каждого файла
    vector = img2vec.get_vec(image_path) # получаем векторное представление
    image_vectors[image_path] = vector # добавляем в словарь, название=ключ.
```

📄 1/1 6s 6s/step
1/1 0s 21ms/step
1/1 0s 21ms/step
1/1 0s 21ms/step
1/1 0s 25ms/step
1/1 0s 21ms/step
1/1 0s 22ms/step
1/1 0s 27ms/step
1/1 0s 25ms/step
1/1 0s 21ms/step
1/1 0s 21ms/step
1/1 0s 20ms/step
1/1 0s 21ms/step
1/1 0s 20ms/step
1/1 0s 21ms/step
1/1 0s 21ms/step
1/1 0s 20ms/step
1/1 0s 22ms/step
1/1 0s 21ms/step
1/1 0s 28ms/step
1/1 0s 21ms/step
1/1 0s 20ms/step
1/1 0s 28ms/step
1/1 0s 26ms/step
1/1 0s 19ms/step
1/1 0s 21ms/step
1/1 0s 23ms/step
1/1 0s 20ms/step

```

1/1 ————— 0s 24ms/step
1/1 ————— 0s 20ms/step
1/1 ————— 0s 19ms/step
1/1 ————— 0s 20ms/step
1/1 ————— 0s 21ms/step
1/1 ————— 0s 20ms/step
1/1 ————— 0s 28ms/step
1/1 ————— 0s 28ms/step
1/1 ————— 0s 20ms/step
1/1 ————— 0s 24ms/step
1/1 ————— 0s 21ms/step
1/1 ————— 0s 20ms/step
1/1 ————— 0s 21ms/step
1/1 ————— 0s 20ms/step
1/1 ————— 0s 20ms/step
1/1 ————— 0s 21ms/step
1/1 ————— 0s 21ms/step
1/1 ————— 0s 20ms/step
1/1 ————— 0s 21ms/step
1/1 ————— 0s 22ms/step
1/1 ————— 0s 21ms/step
1/1 ————— 0s 27ms/step
1/1 ————— 0s 25ms/step
1/1 ————— 0s 28ms/step
1/1 ————— 0s 25ms/step
1/1 ————— 0s 21ms/step
1/1 ————— 0s 20ms/step
1/1 ————— 0s 20ms/step
1/1 ————— 0s 20ms/step
1/1 ————— 0s 20ms/step

```

```
# массив только с векторами, размер вектора 2048
```

```
X = np.stack(list(image_vectors.values()))
```

```
pca_50 = PCA(n_components=50) # первые 50 главных компонент
```

```
pca_result_50 = pca_50.fit_transform(X) # и проецируем на них
```

```
print('Cumulative explained variation for 50 principal components: {}'.format(np.sum(pca_50.explained_variar
```

```
print(np.shape(pca_result_50))
```

```
# 2 компоненты TSNE
```

```
tsne = TSNE(n_components=2, verbose=1, n_iter=3000)
```

```
tsne_result = tsne.fit_transform(pca_result_50) # проецируем
```

```

➡ Cumulative explained variation for 50 principal components: 0.7659520506858826
(291, 50)
[t-SNE] Computing 91 nearest neighbors...
[t-SNE] Indexed 291 samples in 0.001s...
[t-SNE] Computed neighbors for 291 samples in 0.124s...
[t-SNE] Computed conditional probabilities for sample 291 / 291
[t-SNE] Mean sigma: 12.819452
[t-SNE] KL divergence after 250 iterations with early exaggeration: 51.055374
[t-SNE] KL divergence after 1400 iterations: 0.337989

```

```
tsne_result_scaled = StandardScaler().fit_transform(tsne_result) # масштабируем
```

Читаем изображения. Порядок такой же как при создании векторов, ведь как вы помните, TSNE работает только с теми векторами на которых и обучался, с другими не может.

```
# читаем изображения
```

```
images = []
```

```
for image_path in image_paths:
```

```
    image = cv2.imread(image_path, 3) # загружаем изображение
```

```
    b,g,r = cv2.split(image)          # разделяем каналы b, g,
```

```
    image = cv2.merge([r,g,b])        # собираем в r, g, b
```

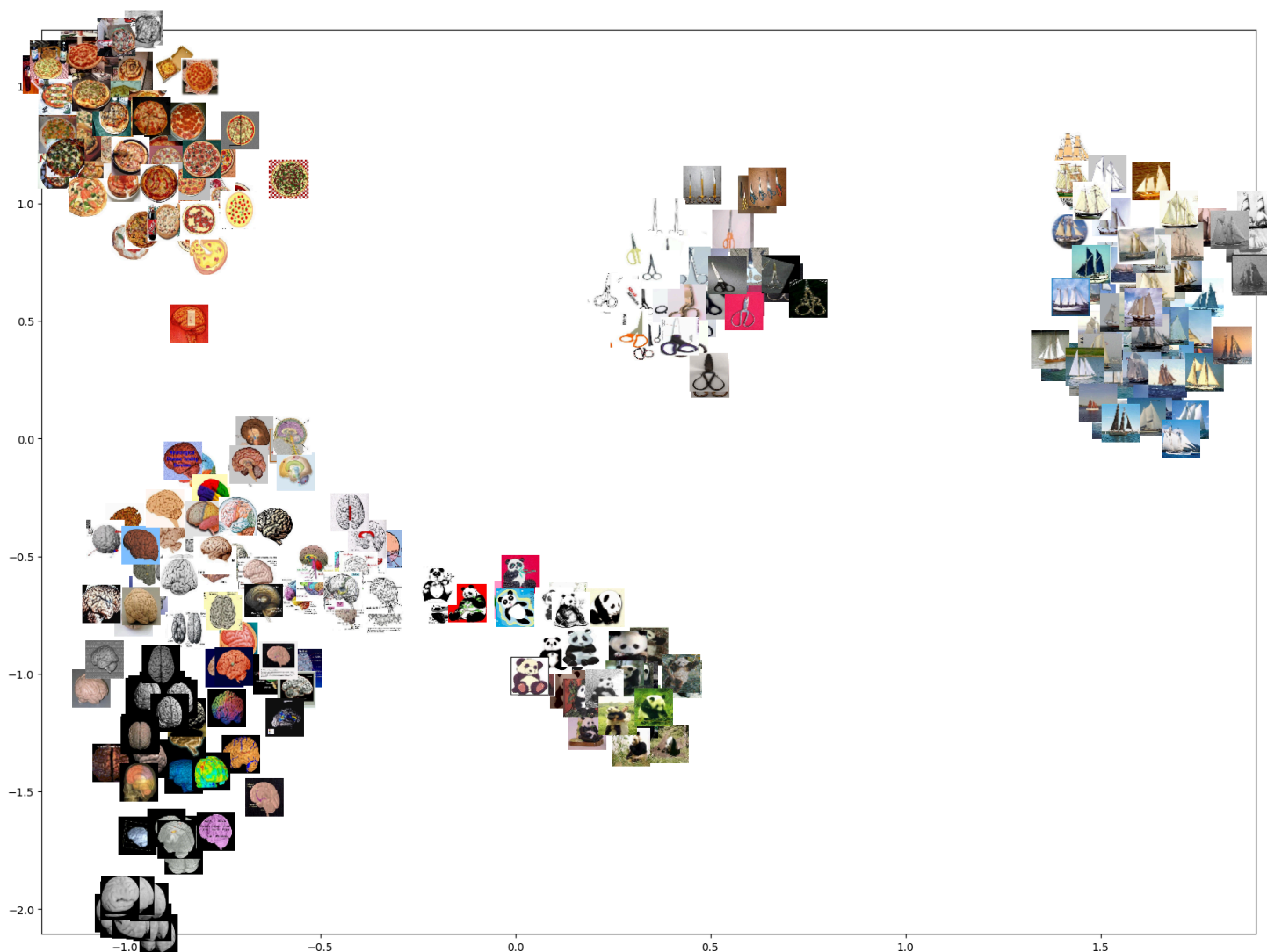
```

image = cv2.resize(image, (50,50)) # масштаб
images.append(image)               # добавляем в список

# рисуем
fig, ax = plt.subplots(figsize=(20,15))
artists = []

for xy, i in zip(tsne_result_scaled, images): # перебираем TSNE вектора (это двумерные координаты) и изображ
    x0, y0 = xy # разделяем горизонтальную и вертикальную координату
    img = OffsetImage(i, zoom=.7) # задаем графический объект с картинкой
    ab = AnnotationBbox(img, (x0, y0), xycoords='data', frameon=False) # вставляем эту картинку по координатам
    artists.append(ax.add_artist(ab)) # добавляем на график
ax.update_datalim(tsne_result_scaled) # пределы отображения
ax.autoscale(enable=True, axis='both', tight=True) # автомасштаб
plt.show() # отображаем

```



И действительно, видим, что похожие изображения собрались в группы. Установили, что у "похожих" изображений "похожие" вектора.

Векторное представление это очень мощный инструмент обработки изображений, теперь мы можем работать с векторами вместо самих изображений. Найти похожие изображения? Нет проблем, сравните их вектора по косинусному расстоянию.

```

# путь к первому изображению
image_path1='101_ObjectCategories/Leopards/image_0001.jpg'

```

```

image_path1='101_ObjectCategories/100/100part_03/image_00001.jpg'
# его вектор, должен быть размером 1на2048
vector1 = np.reshape(img2vec.get_vec(image_path1),(1,-1))

# путь ко второму изображению
image_path2='101_ObjectCategories/Motorbikes/image_0001.jpg'
# его вектор, должен быть размером 1на2048
vector2 = np.reshape(img2vec.get_vec(image_path2),(1,-1))

# их похожесть, чем больше тем похожей
similarity_matrix = cosine_similarity(vector1, vector2)

print('для разных ',similarity_matrix[0][0])

# путь к третьему изображению
image_path3='101_ObjectCategories/Motorbikes/image_0005.jpg'
# его вектор, должен быть размером 1на2048
vector3 = np.reshape(img2vec.get_vec(image_path3),(1,-1))

# их похожесть, чем больше тем похожей
similarity_matrix = cosine_similarity(vector3, vector2)

print('для похожих ',similarity_matrix[0][0])

```

```

1/1 ————— 0s 30ms/step
1/1 ————— 0s 35ms/step
для разных  0.2568908
1/1 ————— 0s 22ms/step
для похожих  0.8999974

```

✓ Ссылки

Использованы и адаптированы материалы:

<https://github.com/jaredwinick/img2vec-keras>